

Java Loops & Jump Statements

Java Loops & Jump Statements

1. Iteration Statements (Loops)
2. The `while` Loop
3. The `do-while` Loop
4. The `for` Loop
5. Nested Loops
6. Jump Statements
7. `break` and `continue` in Nested Loops
8. Labeled Statements — Targeting a Specific Loop

★ One-Page Revision

📝 Practice Questions

🔥 Quick FAQ

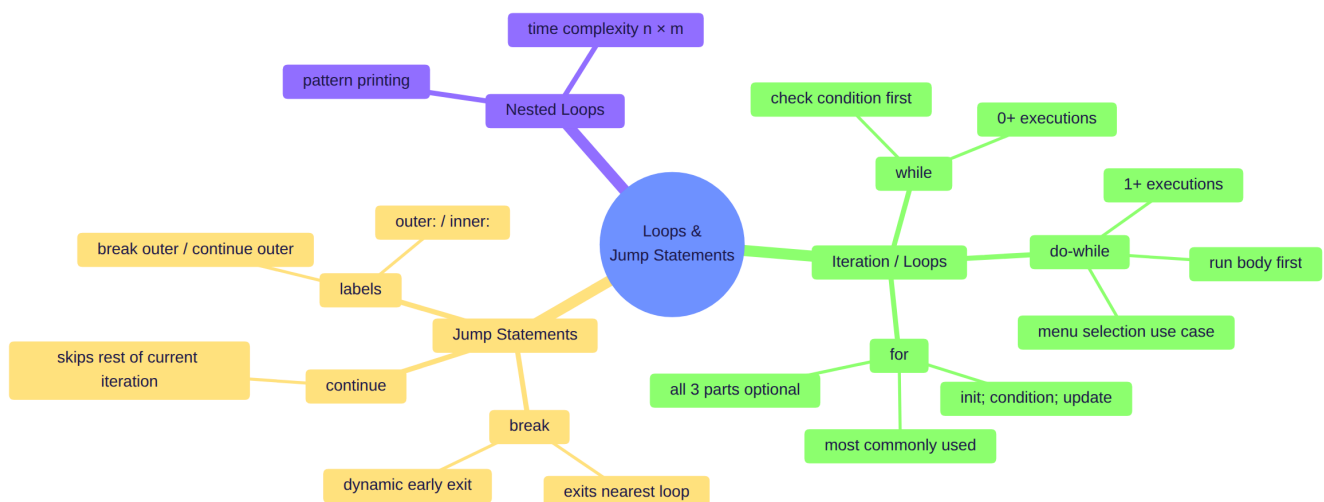
Java Loops & Jump Statements

`while`, `do-while`, `for`, `break`, `continue`, and Labels

1. Iteration Statements (Loops)

Definition: A loop repeats a block of code a specific number of times, or until a condition changes — instead of writing the same statement over and over manually.

Without loops, printing numbers 1 to 10 would require 10 separate `System.out.println()` lines. A loop collapses this into one reusable block that repeats.



Java has **three** loop types, each suited to a different situation:

Loop	Condition Checked	Guarantees
<code>while</code>	Before the body runs	Body may run zero times
<code>do-while</code>	After the body runs	Body runs at least once
<code>for</code>	Before the body runs	Same as <code>while</code> , but packs setup/condition/update into one line

2. The `while` Loop

Syntax:

```
while (expression) {
    // do something
}
```

The `expression` must evaluate to `boolean` — exactly like `if`. As long as it's `true`, the body keeps re-executing; the moment it's `false`, control exits the loop.

Example — print 1 to 10:

```
int i = 1;
while (i <= 10) {
    System.out.println(i);
    i++;
}
```

How it flows: Check `i <= 10` → if `true`, run the body (print `i`, then increment) → jump back to the condition → repeat → stop the instant the condition is `false`.

⚠ **Common Mistake — Infinite Loops:** If you forget the increment (`i++`), the condition `i <= 10` stays `true` forever, and the loop **never terminates** — it keeps printing `1` endlessly. Every loop needs some statement that eventually makes its condition `false`.

Curly Braces and Single Statements

Just like `if`, if the loop body is a **single statement**, braces are optional:

```
while (i <= 10)
    System.out.println(i);
```

🔥 **Important:** In this brace-less form, only the *very next line* belongs to the loop. Writing `i++;` on the following line does **not** include it in the loop body — causing an infinite loop, since `i` never actually changes from the loop's perspective. ✅ **Best Practice:** Always use `{ }` even for one-line bodies, to avoid this exact trap.

Running a Loop in Reverse

```
int i = 10;
while (i >= 1) {
    System.out.println(i);
    i--;
}
```

Swapping `i++` for `i--` (and flipping the starting point/condition) prints `10, 9, 8, ..., 1` instead.

💡 Easy Hinglish Explanation

"**while** loop pehle condition check karta hai, tabhi andar jaata hai. Agar condition pehli baar hi false hai, to loop ek baar bhi nahi chalega."

3. The **do-while** Loop

Syntax:

```
do {
    // do something
} while (expression);
```

The key structural difference from **while**: the body runs **first**, and the condition is checked **afterward** — and only then decides whether to repeat.

Example:

```
int i = 10;
do {
    System.out.println(i);
    i--;
} while (i >= 1);
```

The Critical Difference: Guaranteed First Execution

★ **Exam Tip:** If the condition is **false** from the very start, **while** never runs its body even once — but **do-while** **always** runs the body at least once, since the check happens only *after*.

Proof: Set **i = 11** before a loop checking **i < 10**:

- **while (i < 10) { ... }** → prints **nothing** (condition fails immediately)
- **do { ... } while (i < 10);** → prints **11** once, *then* checks the condition and exits

Real-World Use Case: Menu Selection

do-while is the natural fit whenever something must be **shown to the user at least once**, regardless of any condition — like a game's main menu:

1. Play Game
2. Return to Saved Game
3. Exit

You always want to display this menu once before knowing what the user will pick. Once they choose, you either loop back to the menu again (if they finish a game) or exit (if they chose "Exit"). This is exactly the "do the work first, check after" pattern **do-while** was built for.

💡 Easy Hinglish Explanation

"**do-while** pehle kaam karta hai, condition baad mein check karta hai. Isliye ye kam se kam ek baar to chalega hi chalega — chahe condition shuru se hi false kyun na ho. Menu dikhaane ke liye ye best hai, kyunki user ko option to ek baar dikhana hi hai."

4. The `for` Loop

The most commonly used loop in real code — it packs three separate pieces (initialization, condition, update) into a single line.

Syntax:

```
for (initialization; condition; update) {
    // do something
}
```

Example — print 1 to 10:

```
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

Flow of Control (exact execution order)

1. The **initialization** statement runs — **once only**, before anything else (`int i = 1`)
2. The **condition** is evaluated (`i <= 10`) → `true` or `false`
3. If `true`, the loop **body** executes
4. Once the body finishes, control returns to the `for` statement, and the **update** statement runs (`i++`)
5. The condition is evaluated again
6. Repeat steps 2–5 until the condition is `false`, then exit

📌 **Important:** The initialization step runs only **once** for the entire loop — not on every iteration. Only the condition-check and update repeat.

The `i` Variable's Scope

Declaring `int i` inside the `for(...)` parentheses means `i` only exists **within that loop** — trying to use it afterward gives a compile error ("*i cannot be resolved to a variable*"). This is intentional: since `i` is usually just a loop counter with no purpose outside the loop, keeping it scoped to the loop is the standard, expected practice.

Why `int`, Not `byte` or `short` ?

Even when a loop only needs to count to a small number (like 10), Java developers almost always declare the counter as `int` — never `byte` or `short`.

📌 **Important:** This connects directly back to **type promotion**. Even if you declared `byte i`, Java internally promotes it to `int` for arithmetic anyway — so it consumes the same memory either way. Declaring it as `int` from the start is simply more direct, and has become the universal convention.

Reversing a `for` Loop

```
for (int i = 10; i >= 1; i--) {
    System.out.println(i);
}
```

Multiple Variables — Comma-Separated Initialization/Update

`for` allows **more than one** variable in both the initialization and update sections, separated by commas:

```
for (int i = 1, j = 1; i <= 10; i++, j += 2) {
    System.out.println(i * j);
}
```

Here `i` increases by 1 each time, while `j` increases by 2 — both updating independently within the same loop.

⚠ **Common Mistake:** You **cannot** comma-separate two independent boolean conditions in the middle section (`i <= 10, j <= 5` causes a syntax error — "syntax error on token comma"). To combine multiple conditions, use logical operators instead:

```
for (int i = 1, j = 1; i <= 10 && j <= 5; i++, j += 2) {
    // loop stops the moment EITHER condition fails
}
```

The Condition Can Be Any Boolean Expression

The middle section isn't limited to simple counters — it can be any expression that evaluates to `boolean`, including a variable set elsewhere in the code:

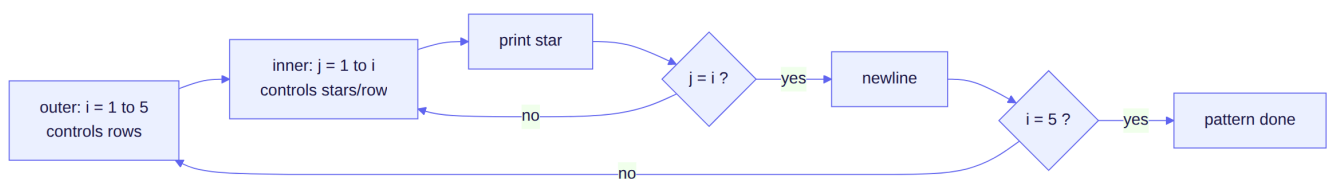
```
boolean b = true;
for (int i = 1; b == true; i++) {
    if (someCondition) {
        b = false; // loop will stop on the next condition check
    }
}
```

5. Nested Loops

A loop can contain another loop inside its body — just like nested `if` or nested `switch`.

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        // inner loop's condition depends on the outer loop's current i
    }
}
```

How execution flows: For **every single iteration** of the outer loop, the **entire** inner loop runs start to finish before the outer loop advances.



Time Complexity

If the outer loop runs `n` times and the inner loop runs `m` times **for each** outer iteration, the total number of inner-body executions is `n × m`.

★ **Exam Tip:** Two nested loops, each running `n` times → **$O(n^2)$** . Three nested loops → **$O(n^3)$** . This generalizes directly — each additional level of nesting multiplies, it doesn't add.

Worked Example: Right-Angle Triangle Pattern

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Output:

```
*
* *
* * *
* * * *
* * * * *
```

Why `j <= i` and not a fixed number? The **outer loop controls rows**, the **inner loop controls how many stars per row**. Making the inner loop's limit depend on `i` (the current row number) is what makes each row longer than the last. If the inner condition were a fixed `j <= 5`, every row would print exactly 5 stars — a rectangle, not a triangle.

 **Important:** `print()` (no `ln`) keeps output on the same line; `println()` at the *end* of the outer loop's body (outside the inner loop) is what moves to a new line after each row completes.

From the Course Files — Heart Pattern

The same exact triangle logic, with a different character, builds any shape:

```
for (int i = 1; i <= 5; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print("❤ ");
    }
    System.out.println();
}
```

 **Best Practice:** Pattern printing is widely used specifically as **practice** for mastering nested loops — the logic of "outer loop = rows, inner loop = depends on outer" generalizes to a huge range of patterns once it clicks.

6. Jump Statements

Definition: Jump statements redirect control flow immediately — either exiting a loop entirely (`break`) or skipping straight to the next iteration (`continue`). Java has two: `break` and `continue` .

 **Important:** `break` isn't exclusive to loops — it's the same keyword used to stop fall-through in `switch` statements. It works identically everywhere: *stop here, don't continue further in this block*.

break — Exit the Loop Immediately

```
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
    if (i > 5) {
        break;
    }
}
```

Output: 1 2 3 4 5 6 — the loop stops completely the instant `i` exceeds 5. Iterations 7 – 10 never happen at all.

★ **Exam Tip:** `break` isn't just for hardcoded numeric conditions — it's most valuable for **dynamically deciding**, mid-execution, that a loop should stop early (e.g. based on a `boolean` flag set somewhere inside the loop body).

Real Example: Prime Number Check

This is the actual code from the lecture — checking whether `p` is prime by testing divisibility, and using `break` to stop early the moment a divisor is found:

```
int p = 7;
int i;
for (i = 2; i < p; i++) {
    if (p % i == 0) {
        System.out.println("The Number is Not Prime");
        break;
    }
}
if (i == p) {
    System.out.println("The Number Is Prime");
}
```

How it works:

- Try dividing `p` by every number from 2 up to (but not including) `p` itself
- The moment any of them divides evenly (`p % i == 0`), the number **cannot** be prime — print the result and `break` immediately, since checking further divisors is pointless
- If the loop finishes *without ever finding* a divisor, `i` will have reached exactly `p` — that's the signal the number is prime

🔴 **Important:** Notice `int i;` is declared **outside** the `for` loop here, not inside `for(int i = 2; ...)`. This is deliberate — the code needs to check `i`'s final value *after* the loop ends, and a variable declared inside `for(...)` only exists within that loop's scope. Declaring it beforehand keeps it accessible afterward.

⚠ **Common Mistake:** Forgetting this scope rule is a common source of the *"i cannot be resolved to a variable"* compile error — if you need a loop counter's value after the loop finishes, declare it outside.

continue — Skip to the Next Iteration

Definition: `continue` skips whatever remains in the **current** iteration and jumps straight back to the loop's next check — it does *not* exit the loop, unlike `break`.

Example — print only odd numbers:

```
for (int i = 1; i <= 10; i++) {
    if (i % 2 == 0) {
        continue;
    }
    System.out.println(i);
}
```

Output: 1 3 5 7 9

How it works: Whenever `i` is even, `continue` fires — skipping the `println` below it entirely for that iteration — and control jumps straight to the next iteration's update/condition check.

💡 Easy Hinglish Explanation

"`break` loop ko पूरी तरह खतम कर deta hai — bahar nikal jaate ho. `continue` sirf current iteration ko skip karta hai — loop chalu rehta hai, bas ye ek round chhod ke agle round pe chala jaata hai."

7. `break` and `continue` in Nested Loops

Critical rule: `break` and `continue` only affect the **nearest enclosing loop** — the one they're physically written inside — not any outer loop.

Example:

```
for (int i = 1; i <= 10; i++) {
    for (int j = 1; j <= i; j++) {
        System.out.print(" * ");
        if (j >= 5) {
            break;
        }
    }
    System.out.println();
}
```

Since `break` sits inside the **inner** loop, it only ever stops that inner loop — the outer loop keeps running normally. The result: every row that would have had 5+ stars gets **capped at exactly 5**, since the inner loop breaks out right when `j` hits 5 — but the outer loop still completes all 10 rows.

Output:

```
*
* *
* * *
* * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

Rows 6 through 10 all look identical — capped at 5 stars — because the inner loop can never get past `j = 5` before breaking.

8. Labeled Statements — Targeting a Specific Loop

By default, `break` / `continue` can only reach the loop they're written directly inside. **Labels** let you name a loop and target it explicitly from a nested position — even from several levels deep.

Syntax: Write an identifier followed by a colon, immediately before the loop:

```
outer: for (int i = 1; i <= 10; i++) {
    inner: for (int j = 1; j <= i; j++) {
        System.out.print(" * ");
        if (j >= 5) {
            break outer;
        }
    }
    System.out.println();
}
```

Now `break outer` exits **both** loops entirely — not just the inner one — the instant `j` reaches 5.

Output:

```
*
* *
* * *
* * * *
* * * * *
```

The pattern stops completely after 5 rows, instead of continuing with capped rows like the earlier unlabeled example.

 **Important:** `continue outer` is equally valid — it jumps straight back to the outer loop's next iteration, skipping the rest of the inner loop entirely.

Easy Hinglish Explanation

"Normally `break` sirf sabse paas wale loop se nikaalta hai. Agar tumhe seedha outer loop se bhi bahar nikalna hai, to us loop ko ek naam (label) do — jaise `outer:` — aur likho `break outer` ya `continue outer`."

Bonus: Named Code Blocks (Rarely Used)

Labels aren't exclusive to loops — **any** `{ }` code block in Java can be labeled, even without a loop or `if` around it:

```
first: {
    second: {
        third: {
            System.out.println("Hello");
            break first; // exits all three nested blocks at once
        }
    }
}
```

 **Best Practice:** This is technically valid Java, but has very limited practical use — it exists mainly to show that labels are a general block-naming feature, not something loops invented specially. In real code, you'll see labels almost exclusively on loops.

★ One-Page Revision

Core Definitions

- **while** = checks condition **before** running; may execute **zero** times
- **do-while** = runs body **once**, then checks; guarantees **at least one** execution
- **for** = combines initialization, condition, and update in one line; most commonly used loop
- **Nested loop** = a loop inside another loop; inner loop fully completes for every outer iteration
- **break** = exits the nearest enclosing loop (or switch) immediately
- **continue** = skips the rest of the current iteration, jumps to the next one
- **Label** = a named identifier (**name:**) placed before a loop, letting **break** / **continue** target that specific loop from anywhere nested inside it

Key Rules

- Infinite loops happen when the terminating condition never becomes **false** — always ensure the loop variable actually changes
- **for** loop's initialization runs exactly **once**; condition and update repeat every cycle
- A **for** loop's counter variable is scoped to the loop unless declared outside it beforehand
- Loop counters are conventionally declared as **int**, never **byte** / **short** — type promotion makes the memory savings moot
- Multiple variables in a **for** loop use **commas** in init/update, but **logical operators** (**&&** , **||**) — never commas — to combine conditions
- **break** / **continue** without a label only affect the **nearest enclosing loop**
- Nested loop time complexity: $n \times m$ for two levels ($O(n^2)$ if both run **n** times), multiplying further with each added level



Practice Questions

Basic

1. What's the key difference between **while** and **do-while**?
2. What does **break** do, and how is it different from **continue**?
3. Write the three parts of a **for** loop's syntax, in order.

Medium 4. Why does a **do-while** loop always execute its body at least once, even if the condition is **false** from the start? 5. Given a nested loop with an unlabeled **break** inside the inner loop, does it stop the outer loop too? Why or why not? 6. Why must **int i;** sometimes be declared *outside* a **for** loop instead of inside it?

Interview 7. Why is **int** the conventional choice for loop counters instead of **byte** or **short**, even when the range would easily fit? 8. Explain, using the prime-number-check example, exactly how and why **break** is used to optimize the check. 9. What problem do labeled **break** / **continue** solve that regular **break** / **continue** cannot? 10. What is the time complexity of three nested loops, each running **n** times, and why?



Quick FAQ

Can a **for loop have empty initialization/condition/update sections?** Yes — all three parts are technically optional, though a completely empty condition creates an infinite loop unless something like a **break** handles termination inside the body.

Is `do-while` used often in real code? Less often than `while` / `for`, but it's the natural fit for menu-driven flows and any "must run at least once" scenario.

Does `continue` skip the loop's update statement (like `i++`) too? No — in a `for` loop, `continue` skips straight to the update step before re-checking the condition; it doesn't bypass the update itself.